National University of Computer and Emerging Sciences



Lab Manual 08
Computer Organization
and Assembly Language

Course Instructor	Mr. Salman Mubarik
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives: 3

Activities: 3

Activity 1 (Assembly Language Tasks): 3

1) Sum of two numbers via stack ..... **Error! Bookmark not defined.**
2) Calculating Power ..... **Error! Bookmark not defined.**
3) Square using stack ..... **Error! Bookmark not defined.**
4) Factorial using local variables ..... **Error! Bookmark not defined.**
5) Transpose of a matrix ..... **Error! Bookmark not defined.**
6) Define Characters ..... **Error! Bookmark not defined.**
7) Copy String ..... **Error! Bookmark not defined.**
8) String Reverse Using LODSB and STOSB 3
9) Substring Search Using SCASB / CMPSB 4
10) Display String Using REP MOVSB 4
11) Scrolling Banner Animation 4
12) Bouncing Character Animation 4
13) Printing Rectangle ..... **Error! Bookmark not defined.**
14) Screen Fill Using REP STOSW and Colored Text Pattern 4

Objectives:

- Display Memory
- String Instructions

Activities:

Activity 1 (Assembly Language Tasks):

1) *Character at the top-left corner of the screen*

   Write a program to place the character 'A' at the top-left corner of the screen (row 0, column 0).

   - Display the character 'A' with white text on a blue background.
   - Display 'A' with a Blue Background and Increased Intensity of white text.

   Video memory for text mode starts at the address 0xB800. Each character on the screen is represented by two bytes: the first byte represents the ASCII code of the character, and the second byte represents its color.

   Color attributes: use their HEX accordingly

   0100 = Red ; R
   0010 = Green ; G
   0001 = Blue ; B
   0000 = Black
   0111 = White
   1000 = Intensity

   Combine Intensity with colors where it seems fit.

2) *Character Using STOS*

   Write a Program to Fill Memory with a Character Using STOS

3) *Solid Horizontal Line using STOSW*

   Use STOSW to draw a solid horizontal line of 80 characters across the middle of the screen (row 12).

   Hint: Calculate the starting address of the 12th row in video memory (B800:0000 + 12 * 80 * 2). Set ES to 0B800h and DI to this calculated offset. Put the character/attribute in AX and use REP STOSW with CX set to 80.

   Example:

   Input: Character: '-', Attribute: 0Eh (yellow on black)

   Output: A yellow horizontal line is drawn across the middle of the console screen.

4) *Compare Two Strings*

   Compare two defined strings of the same length to see if they are identical. For example, check if a user-provided password matches a stored password.

   Hint: Point DS:SI to the first string and ES:DI to the second. Set CX to the length of the strings. Use REPE CMPSB. After the instruction, check the zero flag (ZF). If ZF=1, the strings are identical.

   Example:

   Input: String1 = 'P@ssword', String2 = 'P@ssword'

   Output: A message indicating "Strings are equal."

5) *String Reverse Using LODSB and STOSB*

Write a program that reverses the string "AssemblyLab" using LODSB and STOSB.

- Use Direction Flag = 1 (STD) to store characters in reverse order.
- Store the reversed string at label reversed.
- Display both strings (original and reversed) using Display Memory at:
  - Original → row 3, col 10
  - Reversed → row 4, col 10

6) *Substring Search Using SCASB / CMPSB*

Write a program to find whether the substring "NU" exists inside "FASTNUCEFASTNU".

- Use CMPSB or SCASB with appropriate prefixes (REPNE, REPE).
- Store the starting index of the first occurrence in a variable Index.
- Display "FOUND" or "NOT FOUND" at row 6, col 25 on the screen.

7) *Display String Using REP MOVSB*

Display the string "STRING LAB" at row 4, column 25 using REP MOVSB.

- Store the string in the data segment.
- Use MOVSB with the REP prefix to copy bytes from the source string (DS:SI) to the video memory (ES:DI).

Compute DI using offset = (row * 160) + (column * 2)

- For each character, alternate between ASCII byte and color byte.
- Use light cyan on black (attribute = 0B).

Hint: You can predefine a temporary buffer with alternating bytes (character + color) before copying it with REP MOVSB.

8) *Scrolling Banner Animation*

Write a program that scrolls the message "WELCOME TO COAL LAB" horizontally across row 10 of the screen.

- The text should start from column 0 and move rightwards until column 60.
- Use string operations (MOVSB/STOSB) to update positions.
- Add delay using the provided Delay procedure.
- The animation should appear smooth on DOSBox.

9) *Bouncing Character Animation*

Display a single character (e.g., '@') that moves diagonally on the screen.

- Start from the top-left corner (row 0, col 0).
- On hitting any screen border, reverse direction.
- Keep the motion continuous using the Delay subroutine.
- Use direct video memory access and color change at each bounce.

10) *Screen Fill Using REP STOSW and Colored Text Pattern*

Demonstrate the use of REP STOSW by filling a portion of the video memory with a repeating character pattern.

- Display the character '@' repeatedly across row 12 using REP STOSW.
- Set AX = character + color (e.g., mov ax, 0x0E40 → '@' in yellow).
- Use CX = 80 to fill one full row (80 characters).
- This simulates a horizontal line or screen “fill effect”.

After that, write "DISPLAY TEST" diagonally using manual offset updates (not REP STOSW), where each letter has a unique color attribute.

Hint:

- After each diagonal step, increment DI by 162 (160 + 2).
- Alternate color attribute bytes manually for each letter.